DOCUMENTACIÓN TÉCNICA · FASE DE IMPLEMENTACIÓN



Safari Kids

Sistema de Gestión para Peluquería Infantil

VERSIÓN	FECHA	TIPO
1.0.0	Mayo 2026	Full Stack

Confidencial · Uso Interno · Safari Kids © 2026

📋 Tabla de Contenidos

1. Descripción General del Sistema 3
   - 1.1 Alcance y Objetivos 3
   - 1.2 Stack Tecnológico 3
   - 1.3 Arquitectura General 4
2. Configuración del Entorno de Desarrollo 5
   - 2.1 Requisitos Previos 5
   - 2.2 Base de Datos MySQL 5
   - 2.3 Configuración de application.properties 5
3. Diseño de Base de Datos 6
   - 3.1 Entidades y Relaciones 6
   - 3.2 Descripción de Tablas 7
4. Implementación del Backend 10
   - 4.1 Estructura del Proyecto 10
   - 4.2 Dependencias (pom.xml) 11
   - 4.3 Capa de Seguridad JWT 12
   - 4.4 Controladores REST 14
   - 4.5 Inicialización de Datos 18
5. Implementación del Frontend 19
   - 5.1 Estructura del Proyecto 19
   - 5.2 Configuración Vite y Proxy 20
   - 5.3 Sistema de Autenticación 20
   - 5.4 Enrutamiento y Roles 21
   - 5.5 Módulos por Rol 22
6. Funcionalidades Clave Implementadas 24
7. Pruebas y Datos de Demostración 26

1 Descripción General del Sistema

Visión, objetivos y arquitectura de Safari Kids

1.1 Alcance y Objetivos

Safari Kids es un sistema web de gestión integral diseñado para una peluquería infantil. El sistema cubre el ciclo completo de operación del negocio: desde la reserva de citas por parte de los clientes hasta la gestión administrativa de inventario, ventas, compras y reportes.

3	20+	23	60+
ROLES DE USUARIO	ENTIDADES BD	MÓDULOS FRONTEND	ENDPOINTS REST

Los tres actores principales del sistema son:

👑 **Administrador**

Control total: gestión de usuarios, empleados, inventario, reportes, ventas y compras.

✂️ **Empleado**

Visualiza sus citas asignadas y gestiona el estado de las reservas del día.

👤 **Cliente**

Realiza reservas de servicios, consulta su historial y compra productos en la tienda.

1.2 Stack Tecnológico

Capa	Tecnología	Versión	Propósito
Backend	Spring Boot	3.4.1	Framework principal de la API REST
Backend	Java	17 (LTS)	Lenguaje de programación
Backend	Spring Security + JWT	jjwt 0.11.5	Autenticación y autorización

Backend	Spring Data JPA / Hibernate	3.4.1	ORM y acceso a datos
Base de datos	MySQL	8.x	Persistencia relacional
Build	Maven	3.9+	Gestor de dependencias backend
Frontend	React	18.2	Librería de interfaz de usuario
Frontend	Vite	5.2	Bundler y servidor de desarrollo
Frontend	React Router	6.22	Navegación y rutas SPA
Frontend	TanStack Query	5.28	Caché y sincronización de datos
Frontend	Axios	1.6.8	Cliente HTTP
Frontend	Recharts	2.12	Gráficos y reportes visuales
Frontend	React Hot Toast	2.4.1	Notificaciones
Frontend	npm	—	Gestor de paquetes frontend

1.3 Arquitectura General

El sistema utiliza una arquitectura cliente-servidor desacoplada. El backend expone una API REST stateless protegida con JWT; el frontend React consume esa API a través de un proxy Vite en desarrollo.



💡 **Decisión de diseño:** El proxy de Vite redirige todas las llamadas `/api/*` al backend en el puerto 8080. Esto evita problemas CORS en desarrollo y simplifica la configuración de Axios, que usa `baseURL: '/api'` sin necesidad de conocer el host del backend.

2 Configuración del Entorno de Desarrollo

Pasos para levantar el proyecto desde cero

2.1 Requisitos Previos

Herramienta	Versión mínima	Verificación
Java JDK	17	`java -version`
Maven	3.9+	`mvn -version`
Node.js	18+	`node --version`
MySQL Server	8.0+	`mysql --version`
npm	9+	`npm --version`

2.2 Base de Datos MySQL

La base de datos debe crearse manualmente antes de arrancar el backend. Spring Boot creará las tablas automáticamente con `ddl-auto=update`.

```
-- Crear la base de datos (respetar mayúscula en el nombre)
CREATE DATABASE Safari_kids
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

-- Crear usuario de aplicación (recomendado en producción)
CREATE USER 'safari'@'localhost' IDENTIFIED BY '1234';
GRANT ALL PRIVILEGES ON Safari_kids.* TO 'safari'@'localhost';
FLUSH PRIVILEGES;
```

⚠️ **Nota importante:** El nombre de la base de datos es `Safari_kids` con **S mayúscula**. En sistemas Linux, MySQL distingue mayúsculas en los nombres de BD. El driver JDBC se conecta vía TCP a `127.0.0.1:3306` (no `localhost`) para evitar conflictos con el socket Unix y el plugin de autenticación.

2.3 Configuración de application.properties

Archivo ubicado en `backend/src/main/resources/application.properties`:

```
# ── Servidor ──────────────────────────────────────────
server.port=8080
```

```
# ── Base de Datos ──────────────────────────────────────
spring.datasource.url=jdbc:mysql://127.0.0.1:3306/Safari_kids?\
  useSSL=false&serverTimezone=America/Lima&allowPublicKeyRetrieval=true
spring.datasource.username=safari
spring.datasource.password=1234
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# ── JPA / Hibernate ────────────────────────────────────
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.jpa.hibernate.ddl-auto=update      # crea/actualiza tablas automáticamente
spring.jpa.show-sql=true

# ── JWT ────────────────────────────────────────────────
app.jwt.secret=SafariKids$SecretJWT#2024!XyZ987654321Ab
app.jwt.expiration=86400000               # 24 horas en milisegundos

# ── CORS ───────────────────────────────────────────────
app.cors.allowed-origins=http://localhost:5173
```

2.4 Inicio del Proyecto

```
# 1. Clonar repositorio
git clone <repo-url> safari-kids
cd safari-kids

# 2. Arrancar backend
cd backend
mvn spring-boot:run

# 3. Arrancar frontend (en otra terminal)
cd ../frontend
npm install
npm run dev

# La app estará disponible en: http://localhost:5173
```

3 Diseño de Base de Datos

Modelo relacional, entidades y atributos

3.1 Relaciones entre Entidades

Entidad origen	Relación	Entidad destino	Descripción
Usuario	N:1	Rol	Cada usuario tiene un rol (Admin / Empleado / Cliente)
Cliente	1:1	Usuario	Perfil extendido del cliente
Empleado	1:1	Usuario	Perfil extendido del empleado
Reserva	N:1	Cliente	Un cliente puede tener múltiples reservas
Reserva	N:1	Empleado	Empleado asignado automáticamente (menor carga)
Reserva	N:1	Horario	Franja horaria de la cita
Reserva	N:1	Asiento	Silla temática asignada
DetalleReserva	N:1	Reserva	Servicios contratados en la reserva
DetalleReserva	N:1	Servicio	Servicio específico del detalle
ServicioInsumo	N:1	Servicio	Insumos requeridos por servicio
ServicioInsumo	N:1	Insumo	Insumo consumido y cantidad
Producto	N:1	CategoriaProducto	Clasificación del producto
Inventario	1:1	Producto	Stock actual y mínimo por producto
MovimientoInventario	N:1	Producto	Entradas/salidas/ajustes de stock
Venta	N:1	Cliente	Venta de productos al cliente
DetalleVenta	N:1	Venta	Líneas de productos en la venta
DetalleVenta	N:1	Producto	Producto vendido
Compra	N:1	Proveedor	Compra a proveedor externo

DetalleCompra	N:1	Compra	Productos incluidos en la compra
Pago	N:1	Venta	Registro de pago asociado a venta

3.2 Descripción de Tablas Principales

Usuario

tabla: Usuario · 18 instancias de prueba

PK	idUsuario	INT AUTO	Identificador único del usuario
	nombres	VARCHAR(100)	Nombres del usuario
	apellidos	VARCHAR(100)	Apellidos del usuario
	correo	VARCHAR(100)	Email único, usado como username en JWT
	contrasena	VARCHAR(255)	Hash BCrypt de la contraseña
	estado	ENUM	Activo / Inactivo
	fechaRegistro	DATE	Fecha de creación de la cuenta
FK	id_rol	INT	Referencia a Rol (Administrador / Empleado / Cliente)

Reserva

tabla: Reserva · núcleo del negocio

PK	idReserva	INT AUTO	Identificador único de la reserva
FK	id_cliente	INT	Cliente que realiza la reserva
FK	id_empleado	INT	Empleado asignado (balanceo de carga automático)
FK	id_horario	INT	Franja horaria seleccionada
FK	id_asiento	INT	Silla temática asignada automáticamente
	fechaReserva	DATE	Fecha de la cita
	estado	ENUM	Pendiente / Confirmado / Cancelado
	observaciones	VARCHAR(200)	Notas adicionales del cliente

ServicioInsumo

tabla: Servicio_Insumo · conecta servicios con materiales

PK	idServicioInsumo	INT AUTO	Identificador único
FK	id_servicio	INT	Servicio que consume el insumo
FK	id_insumo	INT	Insumo requerido (shampoo, gel, etc.)
	cantidad	DECIMAL(10,3)	Cantidad consumida por aplicación del servicio

Producto

tabla: Producto

PK	idProducto	INT AUTO	Id ú
	nombre	VARCHAR(100)	Nombre del producto
	precioVenta	DECIMAL(10,2)	Precio al cliente
	costo	DECIMAL(10,2)	Costo d adquisi
	estado	ENUM	Activo / Inactivo
FK	id_categoria	INT	C d p

Inventario

tabla: Inventario

PK	idInventario	INT AUTO	Id ú
FK	id_producto	INT	P (r 1:
	stockActual	INT	Unidad disponi
	stockMinimo	INT	Alerta d reabast
	ubicacion	VARCHAR(100)	Ubicaci física e almacé

4 Implementación del Backend

Spring Boot · API REST · Seguridad JWT

4.1 Estructura del Proyecto Backend

```
backend/src/main/java/com/safarikids/ ├── config/ │ ├── CorsConfig.java # Configuración CORS para el frontend │ ├── DataInitializer.java # Siembra datos iniciales al arrancar │ └── SecurityConfig.java # Spring Security + JWT + reglas de acceso ├── controller/ # 20 controladores REST │ ├── AuthController.java # /api/auth/login y /register │ ├── ReservaController.java # Reservas con asignación automática │ ├── ServicioController.java # Servicios + gestión de insumos │ ├── VentaController.java # Ventas con actualización de stock │ └── ... (16 más) ├── dto/ │ ├── ApiResponse.java # Wrapper estándar: {success, message, data} │ ├── LoginRequest.java │ ├── LoginResponse.java │ └── RegisterRequest.java ├── model/ # 20 entidades JPA │ ├── Usuario.java │ ├── Reserva.java │ ├── ServicioInsumo.java # Tabla puente Servicio ↔ Insumo │ └── ... (17 más) ├── repository/ # 20 interfaces JpaRepository ├── security/ │ ├── JwtFilter.java # Intercepta cada request, valida Bearer token │ ├── JwtUtil.java # Genera y valida tokens HMAC-SHA256 │ └── UserDetailsServiceImpl.java └── service/ # Lógica de negocio para inventario, ventas, compras
```

4.2 Dependencias Clave (pom.xml)

```
<dependencies>
  <!-- API REST -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Seguridad -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>

  <!-- JPA + MySQL -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>
```

```
    <!-- JWT (JJWT 0.11.5) -->
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-api</artifactId>
      <version>0.11.5</version>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-impl</artifactId>
      <version>0.11.5</version>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-jackson</artifactId>
      <version>0.11.5</version>
      <scope>runtime</scope>
    </dependency>

    <!-- Lombok (reduce boilerplate) -->
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
  </dependencies>
```

4.3 Capa de Seguridad JWT

4.3.1 Generación y Validación de Tokens (JwtUtil.java)

Los tokens se generan con el algoritmo HMAC-SHA256 usando una clave secreta configurada en properties. Cada token tiene una vigencia de 24 horas.

```
@Component
public class JwtUtil {

    @Value("${app.jwt.secret}")
    private String secret;

    @Value("${app.jwt.expiration}")
    private long expiration;  // 86400000 ms = 24 horas

    public String generateToken(UserDetails userDetails) {
        return Jwts.builder()
            .setSubject(userDetails.getUsername())  // correo del usuario
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + expiration))
```

```
            .signWith(key(), SignatureAlgorithm.HS256)
            .compact();
    }

    public boolean validateToken(String token, UserDetails userDetails) {
        try {
            String username = extractUsername(token);
            return username.equals(userDetails.getUsername()) && !isExpired(token);
        } catch (JwtException | IllegalArgumentException e) {
            return false;
        }
    }
}
```

4.3.2 Filtro JWT (JwtFilter.java)

El filtro se ejecuta en cada request (extiende `OncePerRequestFilter`). Extrae el token del header `Authorization: Bearer <token>`, lo valida y establece el contexto de seguridad.

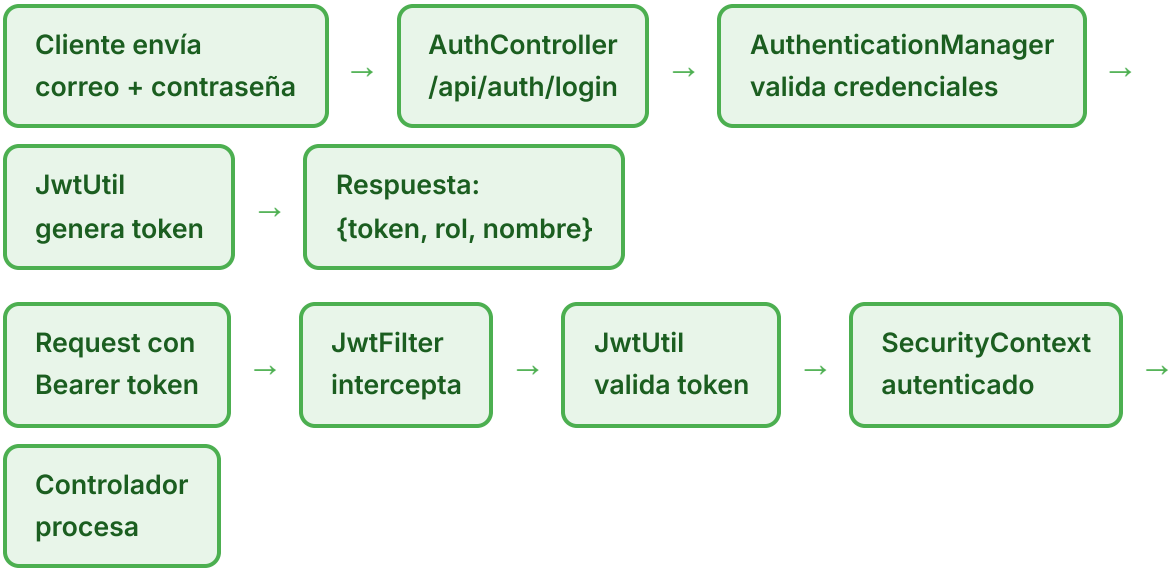
```
@Component
public class JwtFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest req,
                                    HttpServletResponse res,
                                    FilterChain chain) throws ... {
        String header = req.getHeader("Authorization");
        if (header ≠ null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            try {
                String username = jwtUtil.extractUsername(token);
                if (username ≠ null && SecurityContextHolder.getContext()
                                            .getAuthentication() == null) {
                    UserDetails ud = userDetailsService.loadUserByUsername(username)
                    if (jwtUtil.validateToken(token, ud)) {
                        // Establece autenticación en el contexto de seguridad
                        var auth = new UsernamePasswordAuthenticationToken(
                            ud, null, ud.getAuthorities());
                        SecurityContextHolder.getContext().setAuthentication(auth);
                    }
                }
            } catch (JwtException ignored) {} // token inválido: request queda anóni
        }
        chain.doFilter(req, res);
    }
}
```

4.3.3 Configuración de Spring Security (SecurityConfig.java)

```java
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .cors(cors -> {})
        .sessionManagement(sm -> sm.sessionCreationPolicy(STATELESS))
        .exceptionHandling(ex -> ex.authenticationEntryPoint(
            (req, res, e) -> {
                res.setStatus(401);                    // 401 = no autenticado
                res.getWriter().write("{\"error\":\"No autenticado\"}");
            }))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers(GET, "/api/productos/**",
                                  "/api/servicios/**",
                                  "/api/categoria-productos/**",
                                  "/api/horarios/**",
                                  "/api/asientos/**").permitAll()
            .anyRequest().authenticated()
        )
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

4.3.4 Flujo completo de Autenticación



4.4 Controladores REST — Catálogo de Endpoints

Autenticación — `/api/auth`

Método	Ruta	Acceso	Descripción	Body / Response
POST	/api/auth/login	Público	Autenticar usuario	`{correo, contrasena}` → `{token, rol, nombreCompleto}`
POST	/api/auth/register	Público	Registrar nuevo cliente	`{nombres, apellidos, correo, contrasena, telefono}`

Reservas — `/api/reservas`

Método	Ruta	Acceso	Descripción
GET	/api/reservas	Auth	Lista todas las reservas (admin)
GET	/api/reservas/mis-reservas	Auth	Reservas del cliente logueado (por JWT)
GET	/api/reservas/mis-asignadas	Auth	Reservas asignadas al empleado logueado
POST	/api/reservas/crear	Auth	Crear reserva con asignación automática de empleado y asiento
PATCH	/api/reservas/{id}/confirmar	Auth	Confirmar reserva + descuento de insumos
PATCH	/api/reservas/{id}/cancelar	Auth	Cancelar reserva
GET	/api/reservas/rango	Auth	Reservas por rango de fechas (?inicio=&fin=)

Algoritmo de Asignación de Empleado

Al crear una reserva, el sistema asigna automáticamente el empleado con **menor carga del día**:

1. Obtiene todos los empleados con estado `Activo`
2. Cuenta las reservas NO canceladas por empleado para la fecha solicitada
3. Calcula la carga mínima entre todos los activos
4. Filtra los empleados con esa carga mínima
5. Selecciona uno aleatoriamente (desempate justo)

Servicios — `/api/servicios`

Método	Ruta	Acceso	Descripción
GET	/api/servicios	Público	Lista todos los servicios
POST	/api/servicios	Auth	Crear servicio
PUT	/api/servicios/{id}	Auth	Actualizar servicio
DELETE	/api/servicios/{id}	Auth	Eliminar servicio
GET	/api/servicios/{id}/insumos	Auth	Insumos requeridos por el servicio
POST	/api/servicios/{id}/insumos	Auth	Asignar insumo al servicio con cantidad
PUT	/api/servicios/{id}/insumos/{idInsumo}	Auth	Actualizar cantidad de insumo
DELETE	/api/servicios/{id}/insumos/{idInsumo}	Auth	Remover insumo del servicio

Ventas e Inventario — `/api/ventas` · `/api/inventario`

Método	Ruta	Descripción

POST	/api/ventas	Crear venta; descuenta stock automáticamente por cada producto
GET	/api/ventas	Lista todas las ventas
PUT	/api/ventas/{id}/anular	Anula venta y restaura stock
GET	/api/inventario	Estado actual del inventario (stock, mínimos)
POST	/api/inventario/ajuste	Ajuste manual de stock con motivo
GET	/api/inventario/bajo-stock	Productos con stock por debajo del mínimo

Compras — `/api/compras`

Método	Ruta	Descripción
GET	/api/compras	Lista todas las compras a proveedores
POST	/api/compras	Registrar compra; incrementa stock de productos
GET	/api/compras/{id}	Detalle de compra con líneas de productos

Resumen de todos los módulos API

Módulo	Base URL	Acceso GET
Autenticación	`/api/auth`	Público
Usuarios	`/api/usuarios`	Auth
Clientes	`/api/clientes`	Auth
Empleados	`/api/empleados`	Auth
Roles	`/api/roles`	Auth
Servicios	`/api/servicios`	Público
Insumos	`/api/insumos`	Auth
Horarios	`/api/horarios`	Público

Asientos	`/api/asientos`	Público
Reservas	`/api/reservas`	Auth
Productos	`/api/productos`	Público
Categorías	`/api/categoria-productos`	Público
Inventario	`/api/inventario`	Auth
Movimientos	`/api/movimientos-inventario`	Auth
Ventas	`/api/ventas`	Auth
Compras	`/api/compras`	Auth
Proveedores	`/api/proveedores`	Auth
Pagos	`/api/pagos`	Auth
Promociones	`/api/promociones`	Auth
Reportes	`/api/reportes`	Auth

4.5 Inicialización de Datos (DataInitializer.java)

Al arrancar la aplicación por primera vez, `DataInitializer` implementa `CommandLineRunner` y verifica si la BD está vacía (`rolRepo.count() == 0`). Si es así, siembra todos los datos iniciales necesarios para operar el sistema.

Entidad	Cantidad	Detalle
Roles	3	Administrador, Empleado, Cliente
Usuarios	5	1 admin, 2 empleados, 2 clientes
Empleados	2	Carlos (Corte, mañana), Lucía (Tintes, tarde)
Clientes	2	Sofía Ríos, Pedro Lara
Servicios	8	Corte Infantil, Tinte, Lavado, Tratamiento, Peinados Creativos, Spa, Pack Completo, Primera Vez
Horarios	12	09:00–12:00 (mañana) y 14:00–17:00 (tarde), en bloques de 30 min
Asientos	6	Sillas temáticas: Girafa, León, Elefante, Tigre, Mono, Cebra
Categorías	4	Shampoo, Estilizado, Tratamientos, Accesorios
Productos	10	Con inventario inicial aleatorio (10–60 unidades)
Proveedores	2	Distribuidora BabyHair SAC, Cosméticos Andinos EIRL

```
@Component
@RequiredArgsConstructor
public class DataInitializer implements CommandLineRunner {

    @Override
    public void run(String... args) {
        if (rolRepo.count() > 0) return; // idempotente: no re-siembra

        // Roles
        Rol rolAdmin = rolRepo.save(Rol.builder()
            .nombre("Administrador").build());
        Rol rolEmp = rolRepo.save(Rol.builder()
            .nombre("Empleado").build());
        Rol rolCli = rolRepo.save(Rol.builder()
            .nombre("Cliente").build());
```

```
        // Usuarios con contraseñas encriptadas con BCrypt
        Usuario uAdmin = usuarioRepo.save(Usuario.builder()
            .correo("admin@safarikids.com")
            .contrasena(encoder.encode("admin123"))
            .rol(rolAdmin).build());

        // ... más usuarios, servicios, horarios, productos ...
    }
}
```

5 Implementación del Frontend

React 18 · Vite · SPA con roles y autenticación JWT

5.1 Estructura del Proyecto Frontend

```
frontend/src/ ├── api/ # Módulos de acceso a la API (uno por entidad) │ ├── axios.js # Instancia Axios con interceptor JWT │ ├── reservas.js │ ├── productos.js │ └── ... (12 módulos) ├── components/ │ ├── layout/ │ │ ├── PublicLayout.jsx # Navbar + outlet para páginas públicas y cliente │ │ └── MainLayout.jsx # Sidebar + navbar para admin y empleado │ └── common/ │ ├── PrivateRoute.jsx # Guarda rutas por rol │ └── AdminRoute.jsx # Guarda rutas sólo Administrador ├── context/ │ ├── AuthContext.jsx # Estado global de autenticación │ └── CartContext.jsx # Carrito de compras ├── pages/ # 23 páginas organizadas por rol │ ├── landing/ # LandingPage, ServiciosPublic │ ├── auth/ # Login, Register │ ├── cliente/ # MiCuenta, MisReservas, MisCompras, NuevaReserva │ ├── dashboard/ # Dashboard (Admin) │ ├── reservas/ # ReservasPage (Admin) │ ├── inventario/ # InventarioPage (Admin) │ ├── ventas/ # VentasPage (Admin) │ ├── compras/ # ComprasPage (Admin) │ ├── reportes/ # ReportesPage (Admin) │ ├── empleado/ # EmpleadoHome, EmpleadoReservas │ └── tienda/ # TiendaPage, CarritoPage └── App.jsx # Definición de rutas con React Router v6
```

Dependencias principales (package.json)

Paquete	Versión	Uso
react / react-dom	18.2	UI y renderizado
react-router-dom	6.22	Routing SPA y protección de rutas
@tanstack/react-query	5.28	Fetching, caché y refetch automático
axios	1.6.8	Peticiones HTTP con interceptores
react-hot-toast	2.4.1	Toasts de notificación
recharts	2.12	Gráficas de barras y líneas en reportes
react-icons	5.0.1	Iconos (librería Lucide)
jspdf + html2canvas	4.2 / 1.4	Exportación de reportes a PDF
dayjs	1.11	Manipulación de fechas
react-hook-form	7.51	Formularios con validación

5.2 Configuración de Vite y Proxy API

```
// vite.config.js
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
        // Todas las rutas /api/* se reenvían al backend Spring Boot
      }
    }
  }
})
```

5.3 Cliente HTTP con Interceptores (axios.js)

La instancia de Axios centraliza toda la comunicación con el backend. El interceptor de request inyecta automáticamente el token JWT en cada petición. El interceptor de response detecta tokens expirados (401) y fuerza el cierre de sesión.

```
import axios from 'axios'

const api = axios.create({ baseURL: '/api' })

// REQUEST: inyectar token JWT automáticamente
api.interceptors.request.use(config => {
  const token = localStorage.getItem('safari_token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

// RESPONSE: detectar sesión expirada y redirigir al login
api.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) {
      localStorage.removeItem('safari_token')
      localStorage.removeItem('safari_user')
      window.location.href = '/login'
    }
    return Promise.reject(err)
  }
```

```
)

export default api
```

5.4 Sistema de Autenticación (AuthContext.jsx)

El contexto de autenticación provee estado global del usuario logueado mediante React Context. Al login/registro guarda el JWT en `localStorage` y carga el perfil extendido según el rol.

```
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null)

  // Restaurar sesión al recargar la página
  useEffect(() => {
    const stored = localStorage.getItem('safari_user')
    if (stored) setUser(JSON.parse(stored))
  }, [])

  const login = async (correo, contrasena) => {
    const { data } = await api.post('/auth/login', { correo, contrasena })
    const { token, nombreCompleto, rol } = data.data

    // Cargar perfil adicional: idCliente o idEmpleado
    const profile = await fetchProfile(rol, token)
    const userData = { correo, nombreCompleto, rol, token, ...profile }

    localStorage.setItem('safari_token', token)
    localStorage.setItem('safari_user', JSON.stringify(userData))
    setUser(userData)
    return userData
  }

  const logout = () => {
    localStorage.removeItem('safari_token')
    localStorage.removeItem('safari_user')
    setUser(null)
  }
}
```

5.5 Enrutamiento y Protección por Rol (App.jsx)

Se usa React Router v6 con componentes `PrivateRoute` que verifican el rol antes de renderizar la página. Si el usuario no está autenticado, redirige a `/login`. Si tiene un rol incorrecto, redirige a su página de inicio.

```
function PrivateRoute({ allowedRoles = [] }) {
  const { user, loading } = useAuth()
  if (loading) return <Loading />
  if (!user) return <Navigate to="/login" />

  const userRole = user.rol?.toUpperCase()
  if (allowedRoles.length > 0 && !allowedRoles.includes(userRole)) {
    if (userRole === 'CLIENTE')      return <Navigate to="/mi-cuenta" />
    if (userRole === 'EMPLEADO')    return <Navigate to="/empleado" />
    return <Navigate to="/dashboard" />
  }
  return <Outlet />
}
```

Mapa de Rutas por Rol

Ruta	Componente	Rol requerido	Descripción
`/`	LandingPage	Público	Página principal con servicios, testimonios y CTA
`/servicios`	ServiciosPublic	Público	Catálogo de servicios desde la API
`/tienda`	TiendaPage	Público	Tienda de productos con carrito
`/login`	Login	Público	Formulario de acceso
`/registro`	Register	Público	Registro de nuevo cliente
`/reservar`	NuevaReserva	CLIENTE	Wizard 3 pasos para crear reserva
`/mi-cuenta`	MiCuenta	CLIENTE	Dashboard del cliente

`/mi-cuenta/reservas`	MisReservas	CLIENTE	Historial de reservas del cliente
`/mi-cuenta/compras`	MisCompras	CLIENTE	Historial de compras del cliente
`/carrito`	CarritoPage	CLIENTE	Checkout del carrito
`/dashboard`	Dashboard	ADMINISTRADOR	KPIs, gráficas y resumen del negocio
`/reservas`	ReservasPage	ADMINISTRADOR	Gestión de todas las reservas
`/clientes`	ClientesPage	ADMINISTRADOR	CRUD de clientes
`/empleados`	EmpleadosPage	ADMINISTRADOR	CRUD de empleados
`/servicios-admin`	ServiciosPage	ADMINISTRADOR	CRUD de servicios con insumos
`/inventario`	InventarioPage	ADMINISTRADOR	Stock, alertas y movimientos
`/productos`	ProductosPage	ADMINISTRADOR	CRUD de productos de venta
`/ventas`	VentasPage	ADMINISTRADOR	Registro y listado de ventas
`/compras`	ComprasPage	ADMINISTRADOR	Compras a proveedores
`/reportes`	ReportesPage	ADMINISTRADOR	Reportes exportables a PDF
`/usuarios`	UsuariosPage	ADMINISTRADOR	Gestión de usuarios del sistema
`/empleado`	EmpleadoHome	EMPLEADO	Inicio del empleado con citas del día
`/mis-citas`	EmpleadoReservas	EMPLEADO	Reservas asignadas al empleado

6 Funcionalidades Clave Implementadas

Flujos técnicos de las características principales

6.1 Wizard de Reservas (3 Pasos)

El proceso de reserva guía al cliente en 3 pasos con validación en cada transición:

Paso 1 — Servicio

Grid de servicios cargados desde `GET /api/servicios`. Solo muestra los de estado `Disponible`. Al seleccionar uno, avanza al paso 2.

Paso 2 — Fecha y Sede

Selector de fecha (mínimo: hoy), horarios disponibles desde `GET /api/horarios` y selección de sucursal. Botón «Siguiente» deshabilitado hasta completar los 3 campos.

Paso 3 — Confirmar

Resumen completo + campo de notas opcionales. Al confirmar llama a `POST /api/reservas/crear` con JWT. El empleado y asiento se asignan en el backend.

6.2 Gestión de Inventario con Descuento Automático

El inventario se actualiza automáticamente en dos escenarios:

Al confirmar una Reserva

El endpoint `PATCH /api/reservas/{id}/confirmar` ejecuta:

1. Cambia el estado de la reserva a `Confirmado`
2. Obtiene todos los `DetalleReserva` de la reserva
3. Por cada servicio, consulta los `ServicioInsumo` asociados
4. Descuenta `cantidad` del stock de cada `Insumo` (mínimo 0)

Al registrar una Venta

El endpoint `POST /api/ventas` ejecuta:

1. Crea la cabecera de la venta
2. Por cada línea de producto, crea el `DetalleVenta`
3. Actualiza el `Inventario.stockActual` del producto
4. Registra un `MovimientoInventario` de tipo `Salida`

6.3 Sistema de Carrito de Compras

El carrito se gestiona con React Context (`CartContext`) en memoria del navegador. Al hacer checkout, se llama a `POST /api/ventas` con todos los productos del carrito. El stock se actualiza instantáneamente en el backend.

6.4 Respuesta Estándar de la API

Todos los endpoints retornan el mismo wrapper `ApiResponse<T>` para facilitar el manejo en el frontend:

```
// Respuesta exitosa
{
  "success": true,
  "message": "Reserva creada",
  "data": { "idReserva": 42 }
}

// Respuesta de error
{
  "success": false,
  "message": "Credenciales incorrectas",
  "data": null
}
```

6.5 Relación Servicio ↔ Insumo (Implementada en esta sesión)

La tabla `Servicio_Insumo` fue diseñada e implementada para conectar los servicios del salón con los materiales internos que consumen:

```
// POST /api/servicios/{id}/insumos
// Asignar insumo "Shampoo Infantil" al servicio "Corte Infantil"
{
```

```
  "idInsumo": 1,
  "cantidad": 15.500  // ml por aplicación
}

// Al confirmar una reserva de "Corte Infantil":
// → Insumo "Shampoo Infantil": stock -= 15
```

7 Pruebas y Datos de Demostración

Credenciales y escenarios de prueba del sistema

7.1 Credenciales de Acceso

Rol	Correo	Contraseña	Acceso
Administrador	`admin@safarikids.com`	`admin123`	Dashboard completo, todos los módulos
Empleado	`carlos@safarikids.com`	`emp123`	Citas asignadas (turno mañana)
Empleado	`lucia@safarikids.com`	`emp123`	Citas asignadas (turno tarde)
Cliente	`sofia@gmail.com`	`cli123`	Reservas, tienda, mi cuenta
Cliente	`pedro@gmail.com`	`cli123`	Reservas, tienda, mi cuenta

7.2 Escenarios de Prueba por Rol

Escenario 1 — Reserva completa como Cliente

1. Iniciar sesión con `sofia@gmail.com / cli123`
2. Navegar a **Nueva Reserva**
3. Seleccionar "Corte Infantil" (S/ 25)
4. Elegir fecha futura, horario 10:00 y sede Sede Principal
5. Confirmar → aparece toast de éxito y redirige a Mis Reservas

Escenario 2 — Confirmar reserva como Administrador

1. Iniciar sesión con `admin@safarikids.com / admin123`
2. Navegar a **Reservas**
3. Localizar la reserva en estado "Pendiente"
4. Hacer clic en "Confirmar" → estado cambia a "Confirmado"
5. Verificar en **Inventario** que los insumos del servicio bajaron de stock

Escenario 3 — Venta en tienda con descuento de stock

1. Visitar `/tienda` (sin login)
2. Agregar "Shampoo Infantil 400ml" al carrito
3. Iniciar sesión como cliente e ir al carrito
4. Confirmar compra → se crea la venta en el backend
5. Verificar en Inventario (admin) que el stock bajó en 1

7.3 Problemas Resueltos Durante el Desarrollo

Problema	Causa	Solución
POST /api/reservas/crear retorna 403	Spring Security retornaba 403 (en vez de 401) para requests sin token válido. El interceptor de Axios solo manejaba 401.	Se agregó `AuthenticationEntryPoint` en `SecurityConfig` para retornar 401 explícito ante tokens ausentes o expirados.
Tabla Insumo sin conexión	La entidad `Insumo` existía aislada, sin relación con `Servicio` ni flujo de descuento de stock.	Se creó la entidad `ServicioInsumo` (tabla puente), su repositorio y endpoints CRUD. El descuento de stock se integró al endpoint de confirmación de reserva.
Conexión JDBC fallida en Linux	MySQL en Linux usa socket Unix por defecto; el plugin de auth no acepta contraseña vía CLI.	Cambiar la URL de `localhost` a `127.0.0.1` para forzar conexión TCP. Spring Boot / JDBC funciona correctamente por TCP.
Datos hardcoded en componentes	Horarios y servicios estaban definidos como arrays locales en JSX.	Se migraron a llamadas a `GET /api/horarios` y `GET /api/servicios` usando TanStack Query, y se marcaron como endpoints públicos en SecurityConfig.